

NKI-Phys: physics-engine primitives on Trainium

Oct 10, 2026 · @devesh bhayani

What we built

We built the inner loop of a MuJoCo-style physics engine as four NKI kernels, wrapped them in a small engine, and ran an agent loop where Qwen3-8B writes the kernels and a physics-aware checker grades them. A physics engine native to Trainium lets robot-training simulation run on the same chip as the policy network.

The layout is batched like MJX and Brax: one world per partition row (128 worlds per tile), with N bodies per world along the free axis.

Level	Primitive	What it computes	Physical law it must obey
12	Integrator	Semi-implicit (symplectic) Euler: velocity first, then position with the new velocity	Newton's 2nd law
13	Spring forces	A chain of N masses joined by springs, with neighbours taken from shifted slices	Newton's 3rd law (forces sum to zero)
14	Ground contact	Velocity projection $v1 = \max(vf, -x/DT)$, which solves the floor complementarity problem exactly	No penetration; the floor only pushes, and only where it touches
15	Fused rollout	32 steps of springs + gravity + contact in one launch, state kept in SBUF	Bytes: at most 1.5x the one-read-one-write floor

The engine `nkiphys.World` has two modes. `step()` runs the force kernel and then the contact kernel each step. `rollout()` runs one fused kernel per 32 steps. A kernel the agent writes can replace a shipped one once it passes the checker. Code:

```
repo/projects/02-kernel-agent/ ( phys_levels.py , nkiphys.py ,  
phys_device_bench.py ), with a copy in physics/code/ .
```

The checker: scipy as referee, physics as the language

The checker passes its own tests: 4/4 scipy certifications, 4/4 reference kernels accepted, and 12/12 planted physics bugs named correctly in the NKI simulator.

1. **Same-model numerics.** The kernel (float32) must match a float64 NumPy version of the same discrete update. Tolerance is $1e-4$ of RMS per step and $1e-3$ for the 32-step rollout. Every planted bug misses by more than $1e-2$.
2. **scipy certifies the reference itself** (`phys_levels.py --certify`):
 - Spring forces equal minus the gradient of the spring energy (`scipy.optimize.approx_fprime` , max diff $4e-5$).
 - The integrator converges to `scipy.integrate.solve_ivp` (DOP853, rtol $1e-12$) at the expected first order: measured order 1.01, 1.00.
 - Energy stays bounded over 2,000 steps: 18.1% vs 18.4% deviation in the two halves, so no growth. Explicit Euler reaches $10^{175}x$.
 - The contact projection equals the bounded least-squares QP from `scipy.optimize.lsqr_linear` (max diff $9e-16$).
3. **Physics diagnosis in plain language.** When a kernel is wrong, the checker does three things:
 - It replays the reference with each classic bug planted (explicit Euler, missing reaction force, position-only clamp, flipped gravity, one step only, state not carried) and names the one that matches.
 - It measures laws on the kernel's own output: momentum change vs $DT \cdot (f + m \cdot g)$, net internal force, penetration depth, impulse sign, complementarity, energy created.
 - It **recovers the update rule the kernel actually implemented** by least squares over the physical terms ($v, f_{inv_m}, f, v_{inv_m}, inv_m, x, 1$) and prints that rule next to the real one, with one fix per term that differs.

Example feedback the agent received:

your kernel computes $v1 = 0.01f + 0.0001v_{inv_m} - 0.000981$, but the physics is $v1 = 1v + 0.01f_{inv_m} - 0.0981$. To fix: the old velocity v is missing ...; you multiplied the VELOCITY by inv_m ; it is the FORCE that is multiplied by inv_m ...

Results: correct, optimised, and running on the chip

The fused engine moves 51x less HBM traffic than the per-step engine and gives identical answers. On a real NeuronCore it runs correctly and is 52x faster host-to-host.

Claim	Evidence
Engine is physically correct	128 worlds x 16 masses (a slinky drop onto a floor), 64 steps. Both engines match float64 to 4e-5 of RMS. Deepest body is at 0.0 m, so no penetration. 0 of 128 worlds gain energy (mean 80.1 J falls to 7.5 J, matching the reference). In the contact-free phase the NKI trajectory matches scipy <code>solve_ivp</code> to exactly the method's own discretisation error (3.5e-2 m vs 3.5e-2 m)
Fusion optimisation	Unfused: 128 launches, 4.19 MB of HBM traffic (102x the one-read-one-write floor). Fused: 2 launches, 0.08 MB (2.0x). That is 51x less traffic and 64x fewer launches, counted from every <code>dma_copy</code> in simulation
Runs on Trainium	seat-263, NeuronCore 2. Device outputs match float64 to 1e-5 (pass). Host-to-host for 32 steps: unfused 102 s (64 launches), fused 1.96 s (1 launch) = 52x

Caveat on the 52x: on this SDK's standalone numpy path, each launch costs about 1.6 s host-to-host, so the speedup mostly measures launches removed. Device-only execution time is not isolated; that needs the Neuron profiler and is the next measurement.

Agent results: physics feedback moved every wall

Qwen3-8B did not fully solve any physics level in 8 rounds. But each feedback version moved the wall, and by v4 the model's update law was converging on the correct physics term by term. Setup: our seat's Qwen3-8B, 4 samples per round, at most 8 rounds per level. 0.30 = parses but crashes in the simulator; 0.50 = runs, wrong numbers; 1.00 = solved.

Feedback version	L12 integrator	L13 springs	L14 contact	L15 rollout	What changed
v1	0.30	0.50	0.30	not reached	Physics checker only. 24 attempts on L12 stuck on "a tile is still in HBM" without saying which
v2	0.50	0.30	not run	not run	Quote the line and name every HBM input; flag inputs never read and undefined constants. The model copied the suggested lines verbatim
v3	0.50	0.60	0.50 (second pass)	0.30	Recover the update rule the kernel implemented; quote writes into the output buffer
v4	0.50	not run	0.50	not run	More terms in the recovered rule. L14 runs for the first time and gets "THE FLOOR PULLS: at 830 bodies the contact impulse is negative"

The v4 level-12 trace, as recovered by the checker from the model's own kernel:

Round	The model's velocity update
2	$v_1 = v - 9.8$
3	$v_1 = v - 0.0881$
5-7	$v_1 = v + 0.01f_{inv_m} - 0.0881$
Target	$v_1 = v + 0.01f_{inv_m} - 0.0981$

One stray constant remains: the model passes `operand0=0.01` to an add, which puts +0.01 m/s on every velocity. The checker names the wrong constant but not the line that adds it. The next fix is to point to that line. On level 15 the open wall is a 15-wide spring slice copied into a 16-wide tile, where the current hint ("do not reshape") is misleading.

How this maps to standard physics-engine benchmarks

The standard benchmarks test a finished engine. Our ladder builds and checks the primitives those benchmarks exercise. The next primitives fill the gaps they expose.

Benchmark	What it tests	What we already have	Next primitive
Gazebo benchmark 1 : free-floating rigid body	Analytic trajectory (a parabola under gravity); momentum and energy conservation; sweeps over timestep and model count	Trajectory vs analytic/scipy truth; timestep sweep giving convergence order 1.00; model count = the batch axis (1 to 128 worlds per tile); energy checks	Rigid-body integrator with an inertia tensor and quaternions; check angular momentum in free flight
Gazebo benchmark 2 : triball contact	Three contacts with friction; normal forces vs analytic; energy	Frictionless floor contact as an exact complementarity projection, checked for penetration, push-only and complementarity	Coupled multi-contact QP, $\lambda = \max(0, \lambda - \alpha(A\lambda + b))$; one Tensor Engine matmul per iteration, worlds packed block-diagonally into the 128x128 array; then friction cones with a KKT-residual check
GPUPhysBench (50 CUDA tasks)	Agents writing GPU physics kernels; relative L2 error vs expert code; speedup	The Trainium/NKI analogue on a shared task (mass-spring), plus physics-law checks and feedback that names the bug	Their finding sets our order: agents lag most on constraint solving, collision detection and iterative solvers, so the contact QP comes next

In plain English

- We built four building blocks of a physics engine as programs that run directly on the AWS chip: move bodies forward in time, compute spring forces, stop things falling through the floor, and run 32 steps in one go.
- An independent referee (scipy, the standard scientific library) checks that the blocks obey real physics. Energy is never created, forces come in equal and opposite pairs, and nothing sinks into the floor.
- When the AI writes a wrong block, the checker says which law of physics it broke and how to fix it, the way a tutor would. That moved the AI past walls that a plain "numbers wrong" never did.
- Keeping the simulation on the chip for 32 steps, instead of sending it back after every step, moves 51x less data. On the real chip it ran 52x faster end to end, mostly from launching once instead of 64 times.

Reproduce

Run from `projects/02-kernel-agent/` in a seat pod (Neuron SDK, nki 0.6.0). The certification needs only numpy and scipy.

```
python phys_levels.py --certify      # scipy certifies the physics references
(4/4)
python phys_levels.py --selftest    # 4 reference kernels pass, 12 planted
bugs named
python nkiphys.py                  # engine: both modes, physics checks,
HBM traffic, scipy
NEURON_RT_VISIBLE_CORES=2 python phys_device_bench.py  # on a free
NeuronCore
python agent.py --levels 12,13,14,15 --rounds 8 --samples 4 --context 8192 --
keep-best --repeat 2
```

Logs: `logs/phys_v1.jsonl`, `logs/phys_v2.jsonl`, `logs/phys_v3.jsonl`. Feedback changes per version: `physics/PHYS_FEEDBACK_LOG.md`.